

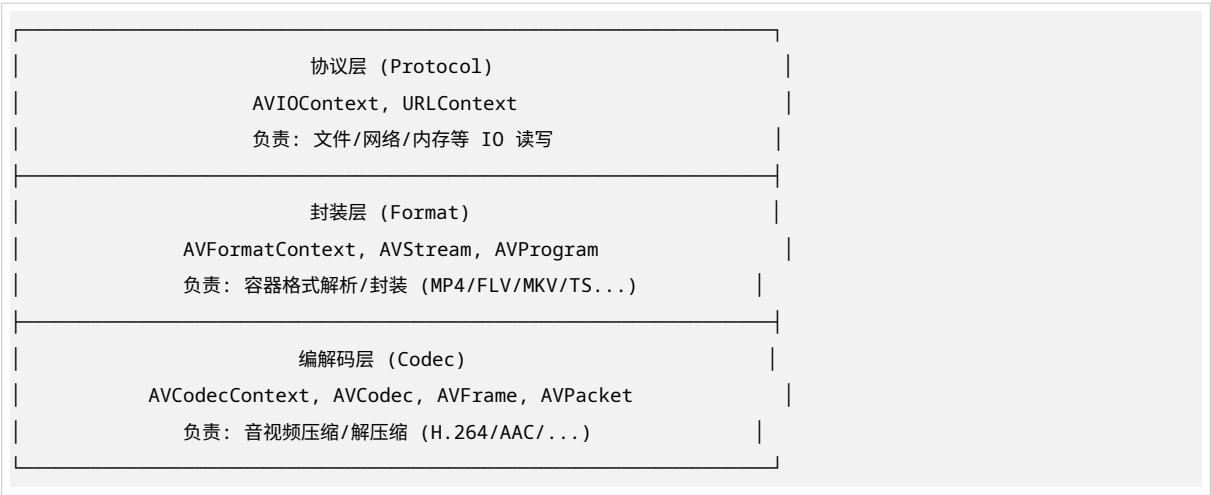
FFmpeg API 协议层/封装层/编解码层常用数据结构深度解析报告

生成时间: 2026-06-24 基于 FFmpeg 编解码实践中关于三层数据结构的讨论整理

目录

- 1. 三层架构总览
- 2. 协议层常用数据结构
- 3. 封装层常用数据结构
- 4. 编解码层常用数据结构
- 5. 三层数据结构关系图
- 6. 数据流转完整示例
- 7. 常用 API 速查表
- 8. 总结

1. 三层架构总览



层级	核心结构体	关联结构体	主要职责
协议层	AVIOContext	URLContext, URLProtocol	字节流读写, 屏蔽底层差异
封装层	AVFormatContext	AVStream, AVProgram, AVCodecParameters	容器格式解析/封装, 流管理
编解码层	AVCodecContext	AVCodec, AVFrame, AVPacket	音视频压缩/解压缩

2. 协议层常用数据结构

2.1 AVIOContext — IO 上下文 (核心)

```
/**
 * 字节流 IO 上下文, 是协议层的核心结构
 * 封装了所有输入输出操作, 对上层屏蔽底层差异
 */
```

```

typedef struct AVIOContext {
    const AVClass *av_class;    // 用于日志和选项

    unsigned char *buffer;      // 内部缓冲区
    int buffer_size;            // 缓冲区大小
    unsigned char *buf_ptr;     // 当前读写位置
    unsigned char *buf_end;     // 缓冲区末尾

    void *opaque;               // 用户自定义数据 (如 FILE*)

    // 读写回调函数指针
    int (*read_packet)(void *opaque, uint8_t *buf, int buf_size);
    int (*write_packet)(void *opaque, uint8_t *buf, int buf_size);
    int64_t (*seek)(void *opaque, int64_t offset, int whence);

    int64_t pos;                // 当前位置 (文件偏移量)
    int eof_reached;            // 是否到达文件末尾
    int write_flag;             // 是否可写
    int max_packet_size;        // 最大包大小 (网络协议用)

    int64_t bytes_read;         // 统计: 已读取字节数
    int64_t bytes_written;      // 统计: 已写入字节数

    // ... 其他字段
} AVIOContext;

```

作用：统一封装文件、网络、内存等各种 IO 操作，上层无需关心底层是本地文件还是 HTTP 流。

```

// 打开文件 IO
AVIOContext *pb;
avio_open(&pb, "input.mp4", AVIO_FLAG_READ);

// 绑定到封装层
AVFormatContext *fmt_ctx = avformat_alloc_context();
fmt_ctx->pb = pb; // 将 IO 上下文交给封装层使用

// 读取数据 (实际通过 pb->read_packet 回调)
avformat_open_input(&fmt_ctx, NULL, NULL, NULL);

```

2.2 URLContext — URL 上下文 (底层)

```

/**
 * URL 上下文, 代表一个具体的协议实例
 * 每个协议 (file/http/rtmp/...) 都有对应的 URLProtocol
 */
typedef struct URLContext {
    const AVClass *av_class;    // 类信息
    const struct URLProtocol *prot; // 协议实现指针
    void *priv_data;           // 协议私有数据

    char *filename;            // URL 地址

```

```
int flags; // 读写标志
int max_packet_size; // 最大包大小
int is_streamed; // 是否为流式 (不可 seek)
int is_connected; // 是否已连接

AVIOInterruptCB interrupt_callback; // 中断回调
} URLContext;
```

URLProtocol 结构 (协议插件接口):

```
typedef struct URLProtocol {
    const char *name; // 协议名: "file", "http", "rtmp"...

    int (*url_open)(URLContext *h, const char *url, int flags);
    int (*url_read)(URLContext *h, unsigned char *buf, int size);
    int (*url_write)(URLContext *h, const unsigned char *buf, int size);
    int64_t (*url_seek)(URLContext *h, int64_t pos, int whence);
    int (*url_close)(URLContext *h);

    int priv_data_size; // 私有数据大小
    const AVClass *priv_data_class;

    int flags; // 协议特性标志
    int (*url_check)(URLContext *h, int mask); // 检查属性
} URLProtocol;
```

常见协议:

协议名	用途	特点
file	本地文件	支持 seek, 最常用
http / https	HTTP 流媒体	支持 range 请求
rtmp / rtmps	实时消息协议	直播推流/拉流
udp / tcp	原始网络传输	低延迟, 无重传
pipe	管道	进程间通信
srtsp	安全 RTP	加密传输
hls	HTTP Live Streaming	苹果流媒体协议

2.3 AVIODirContext / AVIODirEntry — 目录操作

```
// 目录遍历上下文
typedef struct AVIODirContext {
    struct URLContext *url_context;
} AVIODirContext;

// 目录条目信息
typedef struct AVIODirEntry {
    char *name; // 文件名
    int type; // 类型: 文件/目录/符号链接
    int64_t size; // 文件大小
    int64_t modification_timestamp; // 修改时间
```

```

int64_t access_timestamp;    // 访问时间
int64_t status_change_timestamp; // 状态改变时间
int64_t user_id;            // 用户 ID
int64_t group_id;          // 组 ID
int64_t filemode;           // 文件权限模式
} AVIODirEntry;

```

2.4 协议层使用示例

```

// ===== 方式 1: 简单文件 IO =====
AVFormatContext *fmt_ctx = NULL;
avformat_open_input(&fmt_ctx, "input.mp4", NULL, NULL);

// ===== 方式 2: 自定义 IO (内存缓冲区) =====
uint8_t *buffer = av_malloc(4096);
AVIOContext *avio = avio_alloc_context(
    buffer, 4096,           // 缓冲区及大小
    0,                     // write_flag (0= 只读)
    user_data,             // opaque
    my_read_packet,        // read_packet 回调
    NULL,                  // write_packet 回调
    my_seek                // seek 回调
);

AVFormatContext *fmt_ctx = avformat_alloc_context();
fmt_ctx->pb = avio;
avformat_open_input(&fmt_ctx, NULL, NULL, NULL);

// ===== 方式 3: 网络流 =====
AVFormatContext *fmt_ctx = NULL;
avformat_open_input(&fmt_ctx, "http://example.com/stream.flv", NULL, NULL);

// ===== 方式 4: RTMP 推流 =====
AVFormatContext *oc = NULL;
avformat_alloc_output_context2(&oc, NULL, "flv", "rtmp://server/live/stream");
avio_open(&oc->pb, "rtmp://server/live/stream", AVIO_FLAG_WRITE);

```

3. 封装层常用数据结构

3.1 AVFormatContext — 封装上下文（核心）

```

/**
 * 格式上下文，封装层的核心结构
 * 管理一个多媒体文件的所有信息
 */
typedef struct AVFormatContext {
    const AVClass *av_class;    // 用于日志和选项

    // ===== 输入输出 =====

```

```

struct AVInputFormat *iformat;    // 输入格式 (解封装)
struct AVOutputFormat *oformat;  // 输出格式 (封装)

void *priv_data;                  // 格式私有数据 (如 MP4 的 MOVContext)

AVIOContext *pb;                  // IO 上下文指针 (关联协议层)

int ctx_flags;                    // 上下文标志

// ===== 流信息 =====
unsigned int nb_streams;          // 流数量
AVStream **streams;              // 流数组 (视频/音频/字幕...)

unsigned int nb_programs;         // 节目数量 (如 DVB 多节目)
AVProgram **programs;            // 节目数组

char *url;                        // 文件名/URL

// ===== 时间信息 =====
int64_t start_time;              // 起始时间 (AV_TIME_BASE 单位)
int64_t duration;                // 总时长
int64_t bit_rate;                // 总码率
AVRational time_base;            // 全局时间基 (通常 {1, AV_TIME_BASE})

// ===== 元数据 =====
AVDictionary *metadata;          // 元数据键值对 (标题/作者/版权...)

// ===== 包缓冲 =====
AVPacketList *packet_buffer;     // 包缓冲区
AVPacketList *packet_buffer_end;

// ===== 内部状态 =====
int64_t data_offset;             // 数据起始偏移
int raw_packet_buffer_remaining_size;

int max_delay;                   // 最大延迟 (实时流控制)
int flags;                       // 标志位
int probesize;                   // 探测大小
int max_analyze_duration;        // 最大分析时长

// ===== 编解码相关 =====
void *internal;                  // 内部数据
int io_repositioned;             // IO 是否重新定位
AVCodec *video_codec;            // 默认视频编解码器
AVCodec *audio_codec;            // 默认音频编解码器
AVCodec *subtitle_codec;         // 默认字幕编解码器
AVCodec *data_codec;             // 默认数据编解码器

int metadata_header_padding;     // 元数据头填充

// ===== 输出相关 =====

```

```
void *opaque;           // 用户数据
av_format_control_message control_message_cb; // 控制消息回调

int64_t output_ts_offset; // 输出时间戳偏移

// ===== 转储相关 =====
int dump_separator;      // 转储分隔符

// ===== 数据流 ID =====
int data_codec_id;       // 数据流编解码器 ID

// ===== 打开回调 =====
int (*open_cb)(struct AVFormatContext *s, AVIOContext **p, const char *url, int flags, const
↪ AVIOInterruptCB *int_cb, AVDictionary **options);

char *protocol_whitelist; // 协议白名单
char *protocol_blacklist; // 协议黑名单

int max_streams;          // 最大流数
int skip_estimate_duration_from_pts; // 跳过 PTS 估算时长

int max_probe_packets;    // 最大探测包数

// ... 更多字段
} AVFormatContext;
```

核心作用：

功能	说明
管理所有流	streams[] 数组包含所有音视频流
关联 IO	通过 pb 字段连接协议层
格式识别	iformat/oformat 自动识别容器格式
时间控制	管理全局时间基和时长信息
元数据	存储标题、作者等标签信息

3.2 AVStream — 流（核心）

```
/**
 * 流结构体，代表一个具体的音视频流
 * 每个视频/音频/字幕轨道对应一个 AVStream
 */
typedef struct AVStream {
    int index;           // 流索引 (0, 1, 2...)
    int id;              // 流 ID (格式特定, 如 MPEG-TS 的 PID)

    AVCodecContext *codec; // ⚠ 已废弃, 改用 codecpar
    void *priv_data;       // 格式私有数据

    // ===== 时间基 =====
    AVRational time_base;  // 流时间基 (解码/编码后确定)
```

```

// ===== 时间戳 =====
int64_t start_time;          // 流起始时间
int64_t duration;           // 流时长 (以 time_base 为单位)
int64_t nb_frames;          // 帧数 (如果已知)

// ===== 显示信息 =====
int disposition;            // 流属性 (默认/强制/听障/视障...)
AVDiscard discard;          // 丢弃策略 (A/B 帧是否丢弃)

// ===== 采样宽高比 =====
AVRational sample_aspect_ratio; // 采样宽高比 (SAR)

// ===== 元数据 =====
AVDictionary *metadata;     // 流级别元数据 (语言/标题...)

// ===== 平均帧率 =====
AVRational avg_frame_rate;  // 平均帧率 (视频流)

// ===== 包信息 =====
AVPacket attached_pic;      // 附加图片 (如 MP3 封面)

// ===== 编解码参数 (新版推荐) =====
AVCodecParameters *codecpar; // 编解码参数 (替代旧版 codec)

// ===== 内部使用 =====
struct AVFrac pts;          // 当前 PTS (内部使用)

// ===== 时间戳推断 =====
int64_t first_dts;          // 第一个 DTS
int64_t cur_dts;            // 当前 DTS
int64_t last_IP_pts;        // 最后一个 I/P 帧 PTS
int last_IP_duration;       // 最后一个 I/P 帧时长

// ===== 索引 =====
int probe_packets;          // 探测包数
int codec_info_nb_frames;   // 编解码器信息帧数

// ===== 输出相关 =====
int need_parsing;           // 需要解析器
struct AVCodecParserContext *parser;

struct AVPacketList *last_in_packet_buffer;
AVProbeData probe_data;

int64_t pts_buffer[MAX_REORDER_DELAY+1]; // PTS 重排序缓冲
AVIndexEntry *index_entries; // 索引条目 (用于 seek)
int nb_index_entries;        // 索引条目数
int index_entries_allocated_size;

// ===== 实时流 =====

```

```

int64_t interleaver_chunk_size;
int64_t interleaver_chunk_duration;

int request_probe;           // 请求探测
int skip_to_keyframe;        // 跳到关键帧
int skip_samples;           // 跳过采样数

int64_t start_skip_samples; // 起始跳过采样
int64_t first_discard_sample; // 第一个丢弃采样
int64_t last_discard_sample; // 最后一个丢弃采样

int nb_decoded_frames;       // 已解码帧数
int64_t mux_ts_offset;       // 复用时间戳偏移
int64_t pts_wrap_reference; // PTS 环绕参考
int pts_wrap_behavior;       // PTS 环绕行为

// ===== 输出相关 =====
int update_initial_durations_done;
int64_t pts_reorder_error[MAX_REORDER_DELAY+1];
uint8_t pts_reorder_error_count[MAX_REORDER_DELAY+1];

int64_t last_dts_for_order_check;
uint8_t dts_ordered;
uint8_t dts_misordered;

int64_t wrap_bits;           // PTS/DTS 位宽 (通常是 64)

int64_t fifo_size;           // FIFO 大小

// ===== 输出相关 =====
int64_t last_mux_dts;         // 最后复用 DTS
AVRational mux_timebase;      // 复用时间基
int64_t codec_info_duration; // 编解码器信息时长
int64_t codec_info_duration_fields;

int finished_writing_stream; // 流写入完成标志

// ===== 内部 =====
void *internal;               // 内部数据
} AVStream;

```

核心字段详解:

字段	类型	含义
index	int	流在文件中的序号 (0= 视频, 1= 音频...)
time_base	AVRational	流时间基 , 所有该流的 PTS/DTS 以此为单位
avg_frame_rate	AVRational	平均帧率 (视频)
codecpar	AVCodecParameters*	编解码参数 (新版替代 codec)

Table 4 – continued

字段	类型	含义
duration	int64_t	流时长（以 time_base 为单位）
metadata	AVDictionary*	流元数据（如语言 lang=chi）
disposition	int	流属性（默认/强制/字幕类型等）
sample_aspect_ratio	AVRational	采样宽高比 SAR（影响显示比例）

3.3 AVCodecParameters — 编解码参数（新版）

```

/**
 * 编解码器参数，替代旧版 AVCodecContext 的部分功能
 * 用于在封装层和编解码层之间传递参数
 */
typedef struct AVCodecParameters {
    enum AVMediaType codec_type;    // 媒体类型：VIDEO/AUDIO/SUBTITLE/DATA
    enum AVCodecID codec_id;        // 编解码器 ID：H264/AAC/...
    uint32_t codec_tag;             // 编解码器标签（四字码）

    // ===== 视频参数 =====
    int width;                      // 视频宽度
    int height;                     // 视频高度
    AVRational sample_aspect_ratio; // 采样宽高比
    enum AVFieldOrder field_order;  // 场序（逐行/顶场/底场）
    enum AVColorRange color_range;  // 色彩范围（TV/PC）
    enum AVColorPrimaries color_primaries; // 色彩原色
    enum AVColorTransferCharacteristic color_trc; // 传递特性
    enum AVColorSpace color_space;  // 色彩空间
    enum AVChromaLocation chroma_location; // 色度位置

    // ===== 视频额外数据 =====
    uint8_t *extradata;             // 额外数据（如 H.264 的 SPS/PPS）
    int extradata_size;             // 额外数据大小

    // ===== 音频参数 =====
    AVRational framerate;           // 帧率（视频）或采样率分数（音频）
    int format;                     // 像素格式（视频）或采样格式（音频）
    int64_t bit_rate;               // 码率
    int bits_per_coded_sample;      // 每个编码采样位数
    int bits_per_raw_sample;        // 每个原始采样位数

    // ===== 音频参数 =====
    int profile;                    // 编码档次（如 H.264 的 Baseline/Main/High）
    int level;                      // 编码级别

    // ===== 音频特定 =====
    int sample_rate;                // 采样率（音频）
    int channels;                   // 通道数（音频）
    uint64_t channel_layout;        // 通道布局（立体声/5.1...）

    // ===== 音频额外数据 =====

```

```

int block_align;           // 块对齐
int frame_size;           // 帧大小 (每帧采样数)
int initial_padding;      // 初始填充
int trailing_padding;     // 尾部填充
int seek_preroll;         // seek 预卷
} AVCodecParameters;

```

3.4 AVInputFormat / AVOutputFormat — 格式定义

```

// 输入格式 (解封装器)
typedef struct AVInputFormat {
    const char *name;           // 格式名: "mov,mp4,m4a,3gp,3g2,mj2"
    const char *long_name;      // 长名称: "QuickTime / MOV"

    int flags;                  // 标志
    const char *extensions;     // 文件扩展名

    const struct AVCodecTag * const *codec_tag; // 编解码器标签表

    const AVClass *priv_class;  // 私有类 (用于选项)

    // 探测函数
    int (*read_probe)(AVProbeData *);

    // 读取函数
    int (*read_header)(struct AVFormatContext *);
    int (*read_packet)(struct AVFormatContext *, AVPacket *pkt);
    int (*read_close)(struct AVFormatContext *);
    int (*read_seek)(struct AVFormatContext *, int stream_index,
                     int64_t timestamp, int flags);

    int64_t (*read_timestamp)(struct AVFormatContext *s, int stream_index,
                              int64_t *pos, int64_t pos_limit);

    int (*read_play)(struct AVFormatContext *);
    int (*read_pause)(struct AVFormatContext *);

    int (*read_seek2)(struct AVFormatContext *s, int stream_index,
                      int64_t min_ts, int64_t ts, int64_t max_ts, int flags);

    int (*get_device_list)(struct AVFormatContext *s, struct AVDeviceInfoList *device_list);
    int (*create_device_capabilities)(struct AVFormatContext *s, struct AVDeviceCapabilitiesQuery
    ↪ *caps);
    int (*free_device_capabilities)(struct AVFormatContext *s, struct AVDeviceCapabilitiesQuery *caps);
} AVInputFormat;

// 输出格式 (封装器) - 结构类似, 略

```

常见封装格式:

格式	输入/输出	特点
mov,mp4,m4a,3gp	I/O	最常用，支持 H.264/H.265/AAC
flv	I/O	Flash 视频，直播常用
mpepts	I/O	传输流，广播电视标准
matroska,webm	I/O	MKV/WebM，开源格式
avi	I/O	老旧但兼容性好
wav	I/O	无损音频
flac	I/O	无损压缩音频
hls	O	HTTP Live Streaming 输出
dash	O	DASH 流媒体输出
rtsp	I	实时流协议输入

3.5 AVProgram — 节目 (DVB/ATSC 用)

```
/**
 * 节目结构，用于多节目传输流（如 DVB 数字电视）
 * 一个传输流包含多个节目，每个节目有独立音视频流
 */
typedef struct AVProgram {
    int id;                // 节目 ID (service_id)
    int flags;              // 标志
    enum AVDiscard discard; // 丢弃策略

    unsigned int nb_stream_indexes; // 流索引数
    unsigned int *stream_index;     // 流索引数组

    AVDictionary *metadata; // 节目元数据（节目名等）

    int program_num;        // 节目号
    int pmt_pid;            // PMT PID
    int pcr_pid;            // PCR PID

    int64_t pmt_version;    // PMT 版本
    int64_t start_time;     // 起始时间
    int64_t end_time;       // 结束时间

    int64_t pts_wrap_reference; // PTS 环绕参考
    int pts_wrap_behavior;      // PTS 环绕行为
} AVProgram;
```

3.6 封装层使用示例

```
// ===== 解封装（读取）完整流程 =====
AVFormatContext *fmt_ctx = NULL;

// 1. 打开输入
avformat_open_input(&fmt_ctx, "input.mp4", NULL, NULL);

// 2. 获取流信息（解析头部，探测编解码参数）
```

```

avformat_find_stream_info(fmt_ctx, NULL);

// 3. 遍历所有流
for (unsigned int i = 0; i < fmt_ctx->nb_streams; i++) {
    AVStream *stream = fmt_ctx->streams[i];

    // 判断流类型
    if (stream->codecpar->codec_type == AVMEDIA_TYPE_VIDEO) {
        printf("视频流 #d: %dx%d, fps=%.2f\n",
            i,
            stream->codecpar->width,
            stream->codecpar->height,
            av_q2d(stream->avg_frame_rate));
    } else if (stream->codecpar->codec_type == AVMEDIA_TYPE_AUDIO) {
        printf("音频流 #d: %dHz, %d channels\n",
            i,
            stream->codecpar->sample_rate,
            stream->codecpar->channels);
    }
}

// 4. 读取包
AVPacket *pkt = av_packet_alloc();
while (av_read_frame(fmt_ctx, pkt) >= 0) {
    AVStream *stream = fmt_ctx->streams[pkt->stream_index];
    // 处理 packet...
    av_packet_unref(pkt);
}

// 5. 关闭
avformat_close_input(&fmt_ctx);

```

```

// ===== 封装 (写入) 完整流程 =====
AVFormatContext *oc = NULL;

// 1. 分配输出上下文
avformat_alloc_output_context2(&oc, NULL, NULL, "output.mp4");

// 2. 创建视频流
AVStream *video_st = avformat_new_stream(oc, NULL);
video_st->id = oc->nb_streams - 1;
video_st->time_base = (AVRational){1, 25}; // hint

// 3. 设置编解码参数
AVCodecParameters *par = video_st->codecpar;
par->codec_type = AVMEDIA_TYPE_VIDEO;
par->codec_id = AV_CODEC_ID_H264;
par->width = 1920;
par->height = 1080;
par->sample_aspect_ratio = (AVRational){1, 1};

```

```

// 4. 创建音频流
AVStream *audio_st = avformat_new_stream(oc, NULL);
audio_st->time_base = (AVRational){1, 48000};
AVCodecParameters *audio_par = audio_st->codecpar;
audio_par->codec_type = AVMEDIA_TYPE_AUDIO;
audio_par->codec_id = AV_CODEC_ID_AAC;
audio_par->sample_rate = 48000;
audio_par->channels = 2;

// 5. 打开输出文件
avio_open(&oc->pb, "output.mp4", AVIO_FLAG_WRITE);

// 6. 写入头部 (muxer 会覆盖 time_base)
avformat_write_header(oc, NULL);

// 7. 写入帧
av_interleaved_write_frame(oc, pkt);

// 8. 写入尾部
av_write_trailer(oc);

```

4. 编解码层常用数据结构

4.1 AVCodecContext — 编解码器上下文（核心）

```

/**
 * 编解码器上下文，编解码层的核心结构
 * 包含编解码所需的所有参数和状态
 */
typedef struct AVCodecContext {
    const AVClass *av_class;    // 用于日志和选项

    int log_level_offset;       // 日志级别偏移

    enum AVMediaType codec_type; // 编解码类型
    const struct AVCodec *codec; // 编解码器指针
    enum AVCodecID codec_id;     // 编解码器 ID

    unsigned int codec_tag;     // 编解码器标签

    void *priv_data;            // 编解码器私有数据 (如 x264 的 X264Context)

    struct AVCodecInternal *internal; // 内部数据

    void *opaque;               // 用户数据

    // ===== 速率控制 =====
    int64_t bit_rate;           // 目标码率
    int bit_rate_tolerance;     // 码率容差
    int64_t rc_max_rate;        // 最大码率

```

```

int64_t rc_min_rate;          // 最小码率
int rc_buffer_size;          // 码率控制缓冲区大小
float rc_initial_buffer_occupancy; // 初始缓冲区占用

// ===== 视频参数 =====
int width;                    // 视频宽度
int height;                   // 视频高度
int coded_width;              // 编码宽度 (对齐后)
int coded_height;             // 编码高度 (对齐后)
int gop_size;                 // GOP 大小 (关键帧间隔)
enum AVPixelFormat pix_fmt; // 像素格式

// ===== 运动估计 =====
int me_method;                // 运动估计算法
int me_range;                 // 运动估计范围

// ===== 量化参数 =====
int qmin;                     // 最小量化值
int qmax;                     // 最大量化值
int max_qdiff;                // 最大量化差
int rc_qsquish;               // 量化压缩
float rc_qmod_amp;            // 量化调制幅度
int rc_qmod_freq;             // 量化调制频率

// ===== 编码器选项 =====
float rc_initial_cplx;        // 初始复杂度
int trellis;                  // Trellis 量化
float lumi_masking;           // 亮度掩蔽
float temporal_cplx_masking; // 时间复杂度掩蔽
float spatial_cplx_masking; // 空间复杂度掩蔽
float p_masking;              // P 帧掩蔽
float dark_masking;           // 暗部掩蔽

// ===== 预测 =====
int prediction_method;        // 预测方法

// ===== 统计 =====
int64_t scenechange_threshold; // 场景切换阈值
int noise_reduction;          // 降噪强度

// ===== 编码器标志 =====
int me_subpel_quality;        // 子像素运动估计质量
int me_subcmp;                // 子像素比较函数
int mb_cmp;                   // 宏块比较函数
int ildct_cmp;                // iDCT 比较函数
int dia_size;                 // 菱形搜索大小
int last_predictor_count;     // 最后预测器计数

// ===== 预处理器 =====
int pre_me;                   // 预处理运动估计
int pre_dia_size;             // 预处理菱形大小

```

```

int me_pre_cmp;                // 预处理比较函数

// ===== 运动估计参数 =====
int dtg_active_format;        // DTG 活动格式
int me_penalty_compensation;  // 运动估计惩罚补偿

// ===== 编码器选项 =====
enum AVCodecID codec_id;      // 编解码器 ID
int workaround_bugs;          // 兼容性 bug 修复
int strict_std_compliance;    // 标准合规性
int b_frame_strategy;         // B 帧策略
float quantizer_noise_shaping; // 量化噪声整形

// ===== 音频参数 =====
int sample_rate;              // 采样率
int channels;                  // 通道数
enum AVSampleFormat sample_fmt; // 采样格式
int frame_size;               // 帧大小（每帧采样数）
int block_align;              // 块对齐

// ===== 音频通道布局 =====
uint64_t channel_layout;      // 通道布局

// ===== 音频缓冲区 =====
uint8_t *audio_data[AV_NUM_DATA_POINTERS]; // 音频数据指针
int audio_data_size;           // 音频数据大小

// ===== 关键：时间基 =====
AVRational time_base;          // 编解码器时间基

// ===== 关键：帧率 =====
AVRational framerate;          // 帧率（视频）

// ===== 关键：采样宽高比 =====
AVRational sample_aspect_ratio; // 采样宽高比

// ===== 关键：颜色参数 =====
enum AVColorRange color_range;
enum AVColorPrimaries color_primaries;
enum AVColorTransferCharacteristic color_trc;
enum AVColorSpace color_space;
enum AVChromaLocation chroma_location;

// ===== 关键：像素/采样格式 =====
enum AVPixelFormat pix_fmt;      // 视频像素格式
enum AVSampleFormat sample_fmt; // 音频采样格式

// ===== 关键：编解码器选项 =====
AVDictionary *metadata;          // 元数据

// ===== 关键：统计信息 =====

```

```
int64_t rc_override_count; // 码率覆盖数
RcOverride *rc_override; // 码率覆盖数组

// ===== 关键: 量化矩阵 =====
const uint8_t *intra_matrix; // 帧内量化矩阵
const uint8_t *inter_matrix; // 帧间量化矩阵

// ===== 关键: 色度采样 =====
int chroma_sample_location; // 色度采样位置

// ===== 关键: 切片 =====
int slice_count; // 切片数
int *slice_offset; // 切片偏移

// ===== 关键: 场景切换 =====
int scenechange_threshold; // 场景切换阈值

// ===== 关键: 噪声 =====
int noise_reduction; // 降噪

// ===== 关键: Interlaced =====
int interlaced; // 是否交错
int top_field_first; // 顶场优先

// ===== 关键: 压缩级别 =====
int compression_level; // 压缩级别

// ===== 关键: 延迟 =====
int delay; // 编解码延迟

// ===== 关键: 线程 =====
int thread_count; // 线程数
int thread_type; // 线程类型

// ... 大量其他字段, 总计 100+ 个
} AVCodecContext;
```

核心字段分类:

类别	关键字段	说明
尺寸	width, height	视频分辨率
时间	time_base, framerate	时间基和帧率
格式	pix_fmt, sample_fmt	像素/采样格式
音频	sample_rate, channels, channel_layout	音频参数
码率	bit_rate, rc_max_rate, rc_buffer_size	码率控制
质量	qmin, qmax, compression_level	质量控制
GOP	gop_size, max_b_frames	关键帧间隔
线程	thread_count, thread_type	多线程编码

Table 6 – continued

类别	关键字段	说明
----	------	----

4.2 AVCodec — 编解码器定义

```
/**
 * 编解码器结构体，定义一种具体的编解码算法
 * 如 H.264, AAC, HEVC 等
 */
typedef struct AVCodec {
    const char *name;           // 编解码器名: "libx264", "aac", "hevc"
    const char *long_name;      // 长名称: "H.264 / AVC / MPEG-4 AVC"
    enum AVMediaType type;      // 类型: VIDEO/AUDIO/SUBTITLE
    enum AVCodecID id;          // ID: AV_CODEC_ID_H264
    int capabilities;           // 能力标志

    const AVRational *supported_framerates; // 支持的帧率列表
    const enum AVPixelFormat *pix_fmts;     // 支持的像素格式
    const int *supported_samplerates;        // 支持的采样率
    const enum AVSampleFormat *sample_fmts;  // 支持的采样格式
    const uint64_t *channel_layouts;         // 支持的通道布局

    const AVClass *priv_class; // 私有类 (用于选项)

    const AVProfile *profiles; // 支持的档次列表

    // ===== 编解码函数 =====
    int (*init)(AVCodecContext *);
    int (*encode2)(AVCodecContext *avctx, AVPacket *avpkt,
                   const AVFrame *frame, int *got_packet_ptr);
    int (*decode)(AVCodecContext *avctx, void *outdata,
                  int *outdata_size, AVPacket *avpkt);
    int (*close)(AVCodecContext *);

    // ... 其他回调
} AVCodec;
```

常见编解码器:

编解码器	ID	用途	特点
libx264	H264	视频编码	最常用 H.264 编码器
libx265	HEVC	视频编码	H.265/HEVC 高效编码
libvpx-vp9	VP9	视频编码	Google 开源编码器
aac	AAC	音频编码	高级音频编码
libopus	OPUS	音频编码	低延迟高音质
mp3	MP3	音频编解码	兼容性好
flac	FLAC	音频编解码	无损压缩
pcm_s16le	PCM	音频编解码	原始 PCM 数据

4.3 AVFrame — 原始帧

```
/**
 * 原始音视频帧，存储解码后的数据或编码前的原始数据
 */
typedef struct AVFrame {
    uint8_t *data[AV_NUM_DATA_POINTERS];    // 数据指针数组（平面格式用多个）
    int linesize[AV_NUM_DATA_POINTERS];      // 每行字节数

    uint8_t **extended_data;                // 扩展数据指针（通道数 >8 时用）

    int width, height;                      // 视频宽高
    int nb_samples;                          // 音频每帧采样数
    int format;                             // 像素格式或采样格式

    int key_frame;                          // 是否关键帧
    enum AVPictureType pict_type;           // 帧类型：I/P/B...

    // ===== 时间戳 =====
    int64_t pts;                            // 显示时间戳（Presentation Timestamp）
    int64_t pkt_dts;                        // 对应 packet 的 DTS
    AVRational time_base;                   // ⚠ 注意：AVFrame 本身没有 time_base 字段！
                                           // pts 的解释依赖于外部 time_base

    int coded_picture_number;               // 编码序号
    int display_picture_number;             // 显示序号

    // ===== 质量 =====
    int quality;                            // 编码质量

    // ===== 缓冲区 =====
    void *opaque;                          // 用户数据
    uint64_t error[AV_NUM_DATA_POINTERS];   // 错误数组

    int repeat_pict;                        // 重复显示次数

    int interlaced_frame;                   // 是否交错帧
    int top_field_first;                    // 顶场优先

    int palette_has_changed;                 // 调色板是否改变

    int64_t reordered_opaque;               // 重排序 opaque

    int sample_rate;                        // 采样率（音频）
    uint64_t channel_layout;                // 通道布局（音频）

    AVBufferRef *buf[AV_NUM_DATA_POINTERS]; // 缓冲区引用
    AVBufferRef **extended_buf;             // 扩展缓冲区
    int nb_extended_buf;                    // 扩展缓冲区数

    AVFrameSideData **side_data;           // 附加数据数组
```

```
int nb_side_data;          // 附加数据数

int flags;                 // 标志

enum AVColorRange color_range;      // 色彩范围
enum AVColorPrimaries color_primaries; // 色彩原色
enum AVColorTransferCharacteristic color_trc; // 传递特性
enum AVColorSpace color_space;      // 色彩空间
enum AVChromaLocation chroma_location; // 色度位置

int64_t best_effort_timestamp; // 最佳努力时间戳
int64_t pkt_pos;              // 对应 packet 在文件中的位置
int64_t pkt_duration;        // 对应 packet 的时长

AVDictionary *metadata;      // 帧元数据

int decode_error_flags;      // 解码错误标志

int channels;                // 通道数（音频）
int pkt_size;                // 对应 packet 大小

// ===== 硬件加速 =====
AVBufferRef *hw_frames_ctx; // 硬件帧上下文
AVBufferRef *hw_device_ctx; // 硬件设备上下文

// ===== 裁剪 =====
int crop_top, crop_bottom, crop_left, crop_right; // 裁剪区域

AVBufferRef *private_ref;    // 私有引用
} AVFrame;
```

核心字段:

字段	说明
data[] / linesize[]	视频像素数据或音频采样数据
pts	显示时间戳（以 codec time_base 为单位）
width / height	视频分辨率
nb_samples	音频每帧采样数
format	AVPixelFormat 或 AVSampleFormat
key_frame	是否关键帧
pict_type	I/P/B 帧类型

4.4 AVPacket — 压缩数据包

```
/**
 * 压缩音视频数据包，存储编码后的数据
 */
typedef struct AVPacket {
    AVBufferRef *buf;          // 数据缓冲区引用
```

```
int64_t pts;           // 显示时间戳
int64_t dts;           // 解码时间戳
uint8_t *data;         // 数据指针
int size;              // 数据大小
int stream_index;      // 所属流索引

int flags;             // 标志: AV_PKT_FLAG_KEY 等

AVPacketSideData *side_data; // 附加数据
int side_data_elems;      // 附加数据元素数

int64_t duration;      // 时长 (以 stream time_base 为单位)
int64_t pos;           // 在文件中的位置

int64_t convergence_duration; // 收敛时长 (已废弃)
} AVPacket;
```

核心字段:

字段	说明
pts	显示时间戳 (以 stream time_base 为单位)
dts	解码时间戳 (以 stream time_base 为单位)
data / size	压缩数据指针和大小
stream_index	所属流索引
flags	AV_PKT_FLAG_KEY 表示关键帧
duration	包时长 (以 stream time_base 为单位)
pos	在输入文件中的字节位置

4.5 AVCodecParserContext / AVCodecParser — 解析器

```
/**
 * 解析器上下文，用于从原始流中提取完整帧
 * 某些格式 (如 H.264 AnnexB) 需要解析器来分割 NAL 单元
 */
typedef struct AVCodecParserContext {
    void *priv_data;           // 私有数据
    struct AVCodecParser *parser; // 解析器指针

    int64_t frame_offset;      // 当前帧偏移
    int64_t cur_offset;        // 当前偏移
    int64_t next_frame_offset; // 下一帧偏移

    int pict_type;             // 帧类型
    int repeat_pict;           // 重复显示次数
    int64_t pts;               // 当前 PTS
    int64_t dts;               // 当前 DTS

    int64_t last_pts;          // 上一帧 PTS
    int64_t last_dts;          // 上一帧 DTS
};
```

```

int fetch_timestamp;           // 获取时间戳标志

int cur_frame_start_index;     // 当前帧起始索引
int64_t cur_frame_offset[AV_PARSER_PTS_NB]; // 帧偏移数组
int64_t cur_frame_pts[AV_PARSER_PTS_NB];    // PTS 数组
int64_t cur_frame_dts[AV_PARSER_PTS_NB];    // DTS 数组

int flags;                     // 标志

int64_t offset;                // 字节偏移
int64_t cur_frame_end[AV_PARSER_PTS_NB];    // 帧结束数组

int key_frame;                 // 是否关键帧

int64_t convergence_duration; // 收敛时长

int dts_sync_point;            // DTS 同步点
int dts_ref_dts_delta;         // DTS 参考 DTS 差值
int pts_dts_delta;             // PTS DTS 差值

int64_t cur_frame_pos[AV_PARSER_PTS_NB];    // 帧位置数组

int64_t pos;                   // 位置
int64_t last_pos;              // 上一位置

int duration;                  // 时长
enum AVFieldOrder field_order; // 场序

enum AVPictureStructure picture_structure; // 图像结构

// ... 更多字段
} AVCodecParserContext;

```

4.6 编解码层使用示例

```

// ===== 视频解码完整流程 =====
// 1. 查找解码器
const AVCodec *codec = avcodec_find_decoder(AV_CODEC_ID_H264);
if (!codec) { exit(1); }

// 2. 分配上下文
AVCodecContext *codec_ctx = avcodec_alloc_context3(codec);

// 3. 复制参数 (从封装层)
avcodec_parameters_to_context(codec_ctx, stream->codecpar);

// 4. 打开解码器
avcodec_open2(codec_ctx, codec, NULL);

// 5. 分配帧
AVFrame *frame = av_frame_alloc();

```

```

AVPacket *pkt = av_packet_alloc();

// 6. 解码循环
while (av_read_frame(fmt_ctx, pkt) >= 0) {
    if (pkt->stream_index == video_idx) {
        // 发送 packet 到解码器
        int ret = avcodec_send_packet(codec_ctx, pkt);

        // 接收解码后的帧
        while (ret >= 0) {
            ret = avcodec_receive_frame(codec_ctx, frame);
            if (ret == AVERROR(EAGAIN) || ret == AVERROR_EOF) break;
            if (ret < 0) exit(1);

            // frame->pts 可用, frame->data 包含 YUV 数据
            printf("frame %ld, type=%c\n",
                frame->pts,
                av_get_picture_type_char(frame->pict_type));
        }
    }
    av_packet_unref(pkt);
}

// 7. 刷新
avcodec_send_packet(codec_ctx, NULL); // 发送 NULL 刷新
// 继续 receive_frame 直到 EOF

// 8. 清理
av_frame_free(&frame);
av_packet_free(&pkt);
avcodec_free_context(&codec_ctx);

```

```

// ===== 视频编码完整流程 =====
// 1. 查找编码器
const AVCodec *codec = avcodec_find_encoder(AV_CODEC_ID_H264);
if (!codec) { exit(1); }

// 2. 分配上下文
AVCodecContext *codec_ctx = avcodec_alloc_context3(codec);

// 3. 设置参数
codec_ctx->width = 1920;
codec_ctx->height = 1080;
codec_ctx->time_base = (AVRational){1, 25}; // 时间基
codec_ctx->framerate = (AVRational){25, 1}; // 帧率
codec_ctx->pix_fmt = AV_PIX_FMT_YUV420P;
codec_ctx->bit_rate = 4000000; // 4Mbps
codec_ctx->gop_size = 25; // GOP=25
codec_ctx->max_b_frames = 2; // 最大 2 个 B 帧

// 4. 打开编码器

```

```

avcodec_open2(codec_ctx, codec, NULL);

// 5. 分配帧和 packet
AVFrame *frame = av_frame_alloc();
frame->format = codec_ctx->pix_fmt;
frame->width = codec_ctx->width;
frame->height = codec_ctx->height;
av_frame_get_buffer(frame, 0);

AVPacket *pkt = av_packet_alloc();

// 6. 编码循环
for (int i = 0; i < 250; i++) {
    // 填充 YUV 数据...

    frame->pts = i; // 以 time_base 为单位

    // 发送帧
    int ret = avcodec_send_frame(codec_ctx, frame);

    // 接收 packet
    while (ret >= 0) {
        ret = avcodec_receive_packet(codec_ctx, pkt);
        if (ret == AVERROR(EAGAIN) || ret == AVERROR_EOF) break;
        if (ret < 0) exit(1);

        // pkt->pts/dts 可用, 写入文件或处理
        fwrite(pkt->data, 1, pkt->size, outfile);
        av_packet_unref(pkt);
    }
}

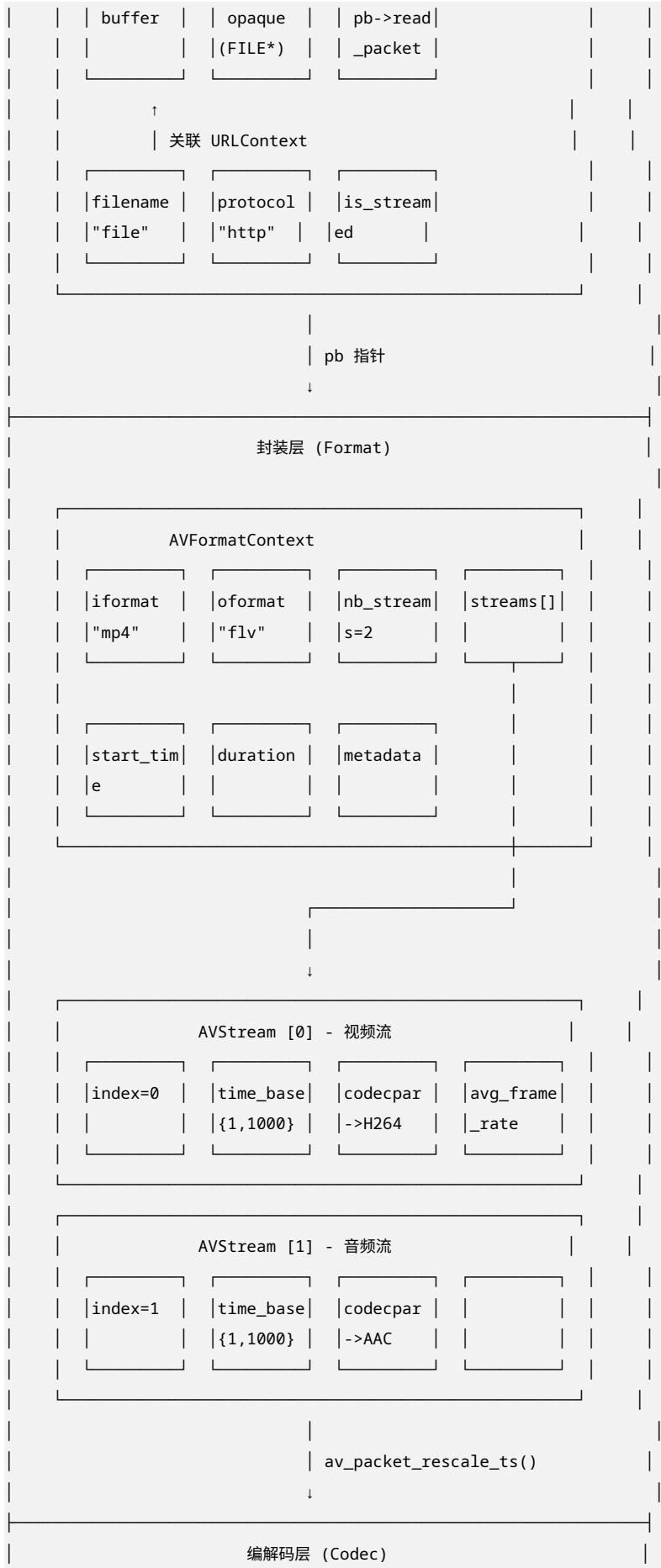
// 7. 刷新
avcodec_send_frame(codec_ctx, NULL);
// 继续 receive_packet 直到 EOF

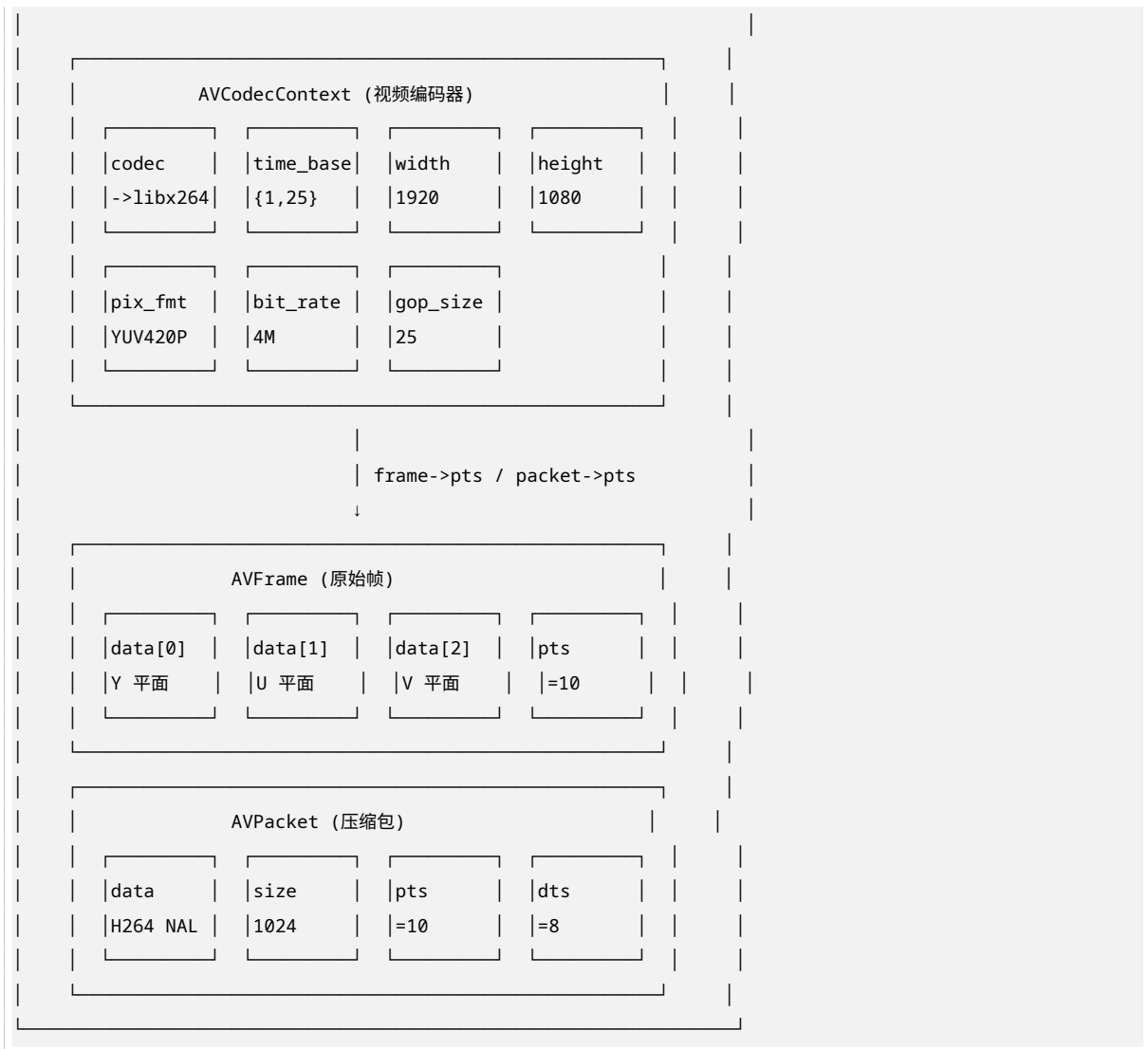
// 8. 清理
av_frame_free(&frame);
av_packet_free(&pkt);
avcodec_free_context(&codec_ctx);

```

5. 三层数据结构关系图



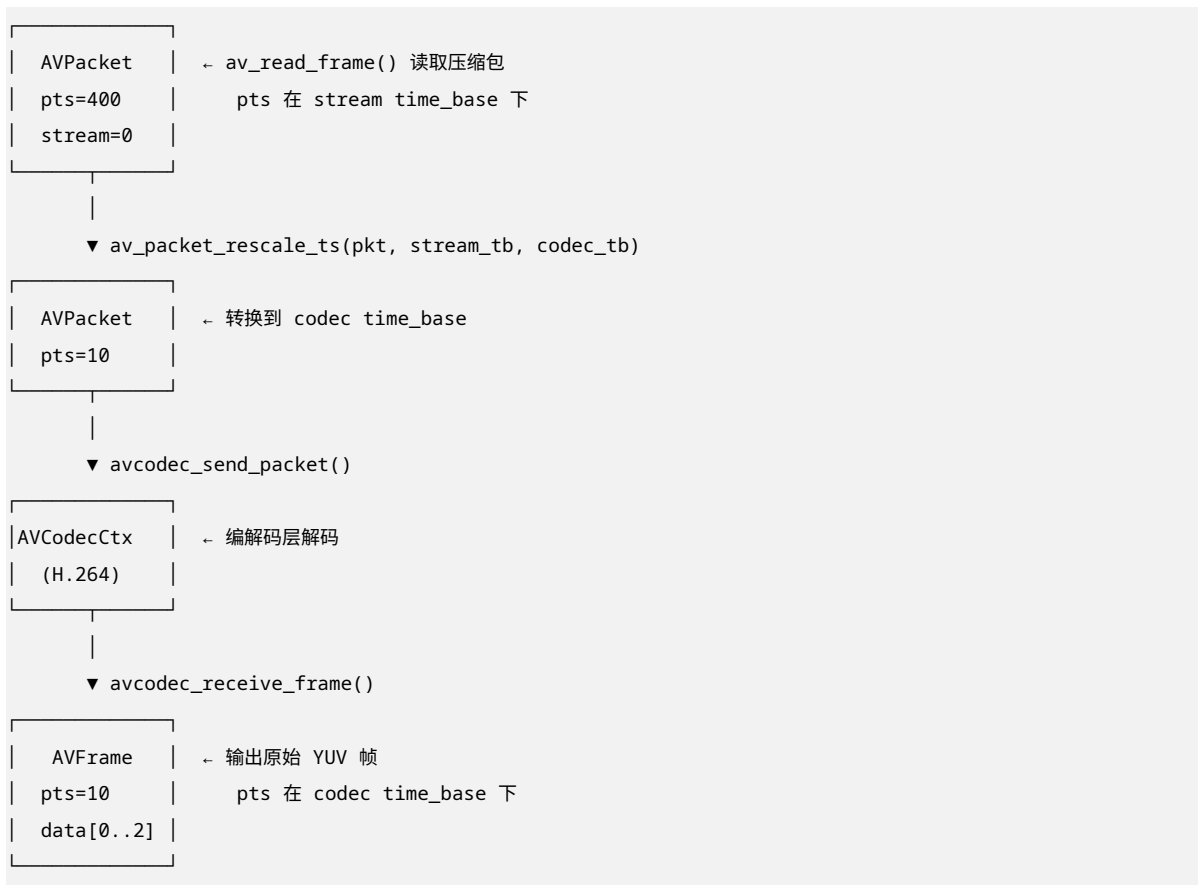




6. 数据流转完整示例

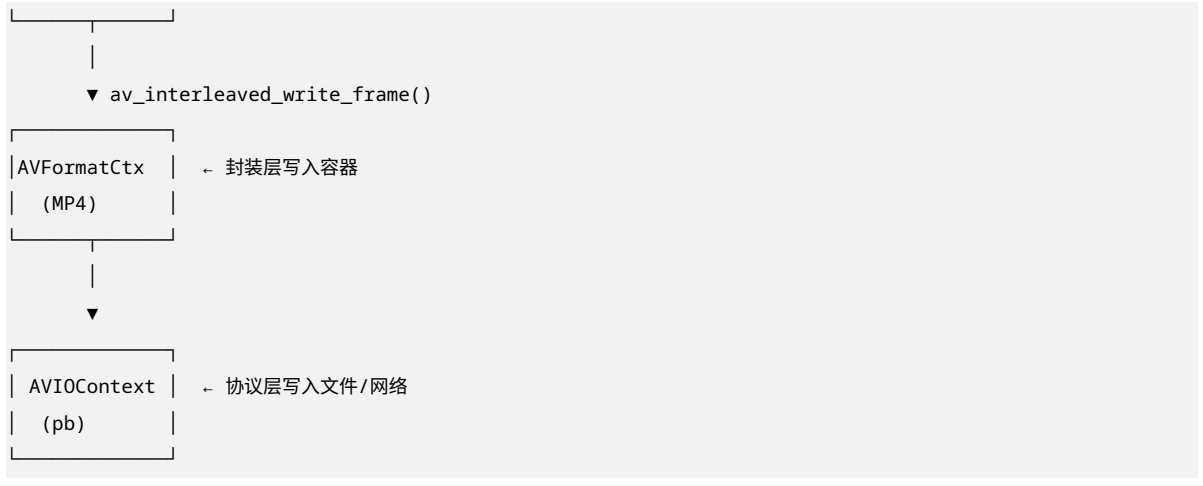
6.1 解封装 + 解码流程





6.2 编码 + 封装流程





7. 常用 API 速查表

7.1 协议层 API

API	功能
avio_open() / avio_close()	打开/关闭 IO
avio_alloc_context()	创建自定义 IO
avio_read() / avio_write()	读写数据
avio_seek()	定位
avio_size()	获取大小

7.2 封装层 API

API	功能
avformat_open_input()	打开输入文件
avformat_find_stream_info()	获取流信息
av_read_frame()	读取一帧
avformat_seek_file()	定位到指定时间
avformat_alloc_output_context2()	创建输出上下文
avformat_new_stream()	创建新流
avformat_write_header()	写入文件头
av_interleaved_write_frame()	交错写入帧
av_write_trailer()	写入文件尾

7.3 编解码层 API

API	功能
avcodec_find_decoder() / avcodec_find_encoder()	查找编解码器
avcodec_alloc_context3()	分配上下文

Table 12 – continued

API	功能
avcodec_open2()	打开编解码器
avcodec_send_packet() / avcodec_receive_frame()	解码 (新 API)
avcodec_send_frame() / avcodec_receive_packet()	编码 (新 API)
avcodec_parameters_to_context()	参数复制到上下文
avcodec_parameters_from_context()	上下文参数复制

8. 总结

8.1 三层数据结构核心职责

层级	核心结构体	关键职责
协议层	AVIOContext	字节流读写，屏蔽文件/网络/内存差异
封装层	AVFormatContext	容器格式解析/封装，流管理，元数据
编解码层	AVCodecContext	音视频压缩/解压缩，参数控制

8.2 关键数据流转原则

- 1. 协议层 → 封装层: 通过 AVFormatContext->pb 指针关联
- 2. 封装层 → 编解码层: 通过 avcodec_parameters_to_context() 传递参数
- 3. 编解码层 → 封装层: 通过 av_packet_rescale_ts() 转换时间基
- 4. 时间基转换: 永远是 实际时间 = pts × time_base, 跨层转换保持实际时间不变

8.3 一句话总结

“协议层管”怎么读写字节流”，封装层管”是什么容器格式”，编解码层管”怎么压缩解压缩”。三层通过指针和 API 紧密协作，时间基转换是贯穿三层的核心机制。

报告结束